

# Трехзвенные приложения

Что такое трехзвенное приложение рассмотрим на примере [Presto](#).

Для того чтобы официанту в ресторане не ходить по залу с терминалом и кассовым аппаратом, ему выдается клиентское устройство, например, смартфон или планшет, который работает оффлайн (без подключения к облаку), но подключается к промежуточному серверу и передает ему информацию о заказах, покупках и т.п. Промежуточный сервер пробивает чек как централизованный пост.



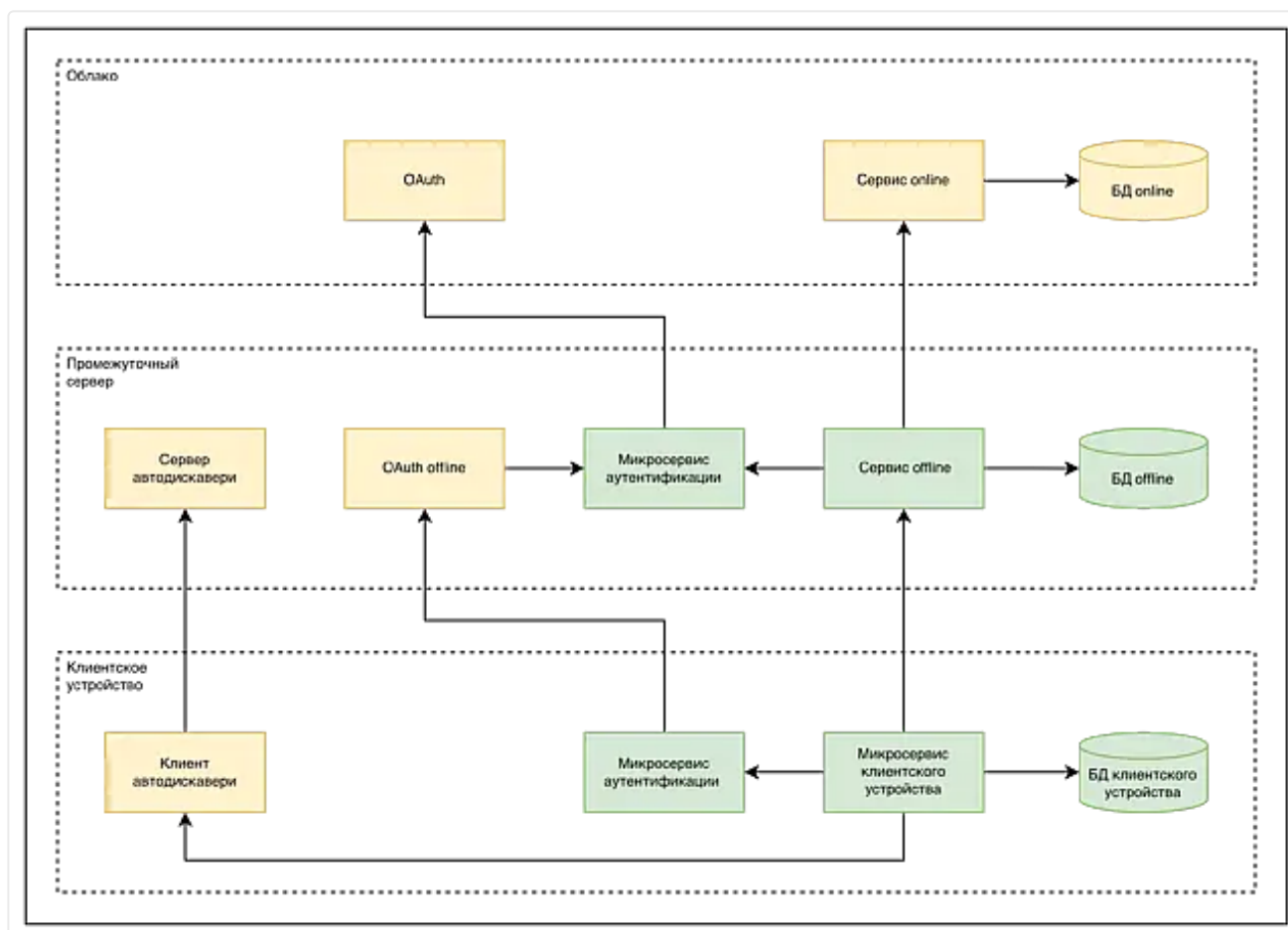
Приведенный выше пример описывает трехзвенную архитектуру приложения, которая состоит из следующих слоев:

- облако – обеспечивает хранение и обработку информации;
- промежуточный (локальный) сервер – собирает сведения с клиентских устройств и передает их в облако, тем самым решает задачу автономной работы;
- клиентские устройства – служат клиентами относительно промежуточного сервера.

Подобная архитектура позволяет решить следующие задачи:

- обеспечение высокой устойчивости к сбоям;
- бесперебойная работа в оффлайн-режиме;
- независимость от обновления облака.

## Архитектура



## Автооткрытие

С помощью [микросервиса автооткрытия](#) клиентское устройство обнаруживает промежуточный сервер и подключается к нему. Если такого сервера нет, то клиентское устройство подключается к облаку.

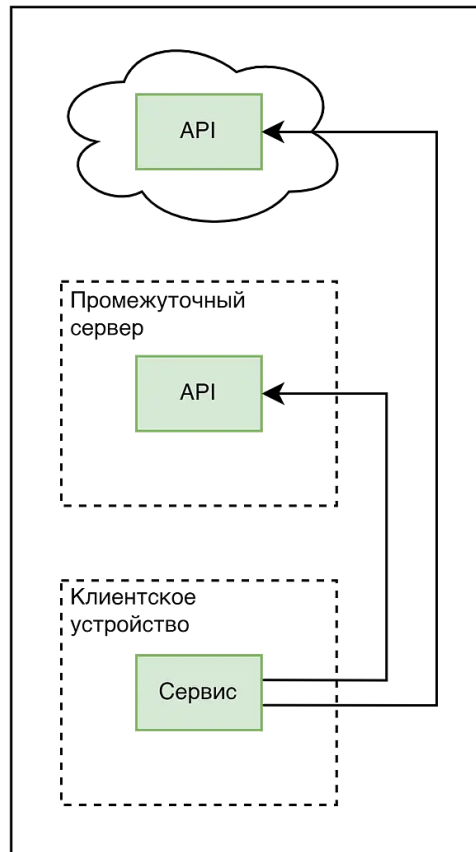
## Аутентификация

Аутентификация в трехзвенной архитектуре организована аналогично [подключению офлайн-приложения к облаку](#). Разница только в том, что на промежуточном сервере должен быть организован OAuth-клиент, который эмулирует для клиентского устройства работу OAuth-клиента в облаке.

Например, если через [oauth\\_service](#) промежуточного сервера клиентское устройство пытается авторизовать пользователя и при этом пользователь еще не авторизован на промежуточном сервере, то происходит процедура аутентификации пользователя в промежуточном сервере с последующей отправкой его данных в клиентское устройство.

## Симметрия API локального сервера и облака

Для работы клиента промежуточный сервер должен предоставлять точно такое же RPC API, что и облако.



## Обновления приложений и совместимость версий

При проектировании многозвенной архитектуры нужно помнить, что обновления промежуточного сервера и клиента происходят асинхронно. А это означает, что могут быть следующие комбинации версий:

1. Старый клиент - старый промежуточный сервер - облако.
2. Старый клиент - новый промежуточный сервер - облако.
3. Новый клиент - новый промежуточный сервер - облако.
4. Новый клиент - старый промежуточный сервер - облако. Эта комбинация является уникальной для трехзвенной архитектуры.

При проектировании изменений разработчик должен предусмотреть корректную работу всех вышеперечисленных сценариев (прямую и обратную совместимость) в течение минимум полугода\*.

\* - согласно пользовательскому соглашению Тензор обязуется в течение полугода поддерживать устаревшие версии автономных приложений.

## Отправка событий

В трехзвенной архитектуре промежуточный сервер выступает единственным [STOMP-сервером](#) для клиента, то есть и в части отправки [событий](#) эмулирует облачную инфраструктуру для клиента.

Схема отправки событий:

